Machine Learning Operations (MLOps)

MLOps-Driven Framework for Carbon Price Forecasting Using Time-Series Models

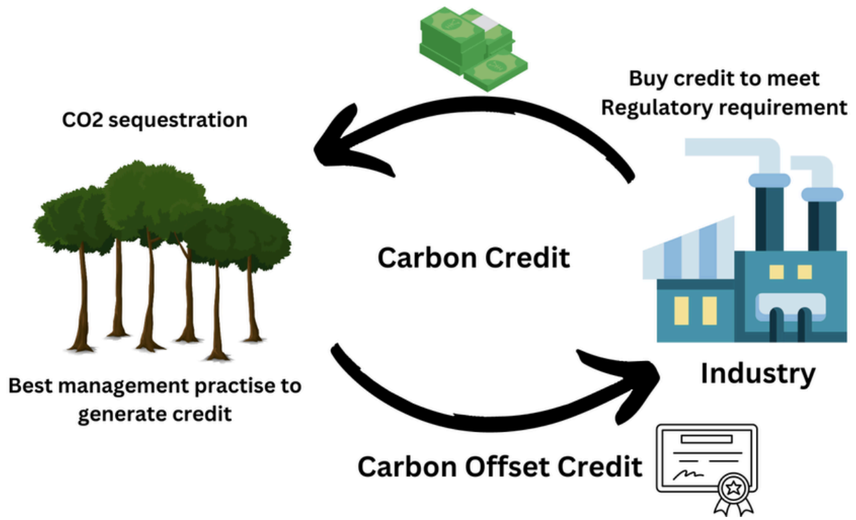
Team No : 2

USN	NAME
1RV23AI065	NAVYASRI M PULIPATI
1RV23AI069	NISHTA N SHETTY
1RV23AI071	NITYA SHARMA
1RV23AI073	PRATHAM M MALLYA

What is Carbon Trading

Carbon trading is a market-based system designed to reduce greenhouse gas emissions by allowing companies to buy and sell carbon credits.

Each carbon credit represents the right to emit one tonne of CO_2.

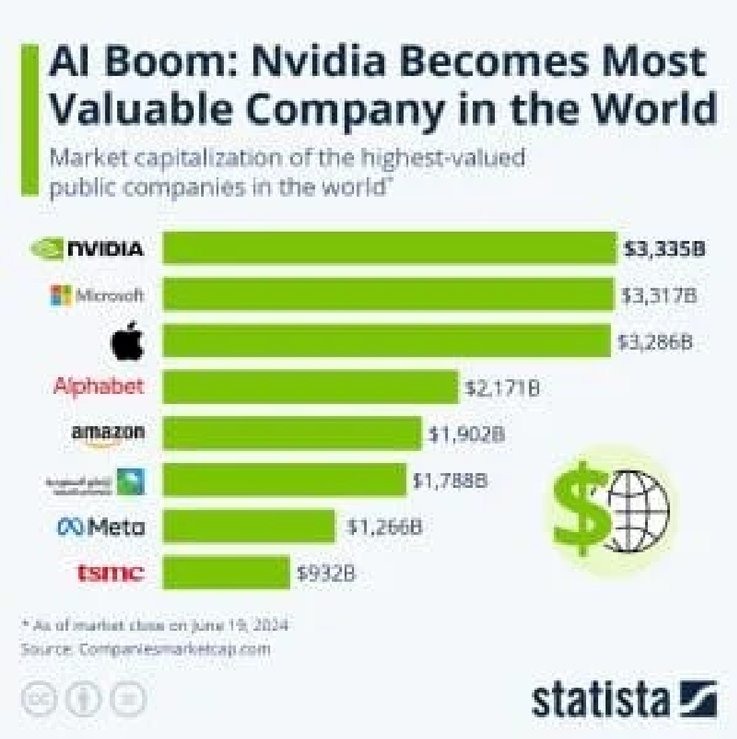


How it Works ?

1. A government sets an emission limit for industries.
2. Company A emits less CO_2 than allowed → earns extra carbon credits.
3. Company B emits more CO_2 → buys credits from Company A.
4. This encourages companies to reduce pollution and adopt cleaner technologies.

Motivation – Example of NVIDIA Training an AI Model

When NVIDIA trains a large AI model, it uses thousands of powerful GPUs running continuously in data centers. These GPUs consume a huge amount of electricity, and most of this electricity still comes from sources that produce carbon emissions



For example, training a large language model may run for several weeks, causing high CO_2 emissions. If NVIDIA exceeds its allowed emission limit, it must buy carbon credits from the carbon trading market. Since carbon prices keep changing, predicting future prices helps NVIDIA decide when to buy credits early at lower cost or reduce emissions using efficient hardware, saving both money and the environment.

Comet ML – Experiment Tracking

What is it ?

Comet ML tracks machine learning experiments and results.

Why to use ?

To compare different carbon price prediction models easily

How it helps ?

When ARIMA and Random Forest are trained on carbon trading data, Comet ML shows which model gives lower error and better forecasts.

Apache Airflow – Workflow Automation

What is it ?

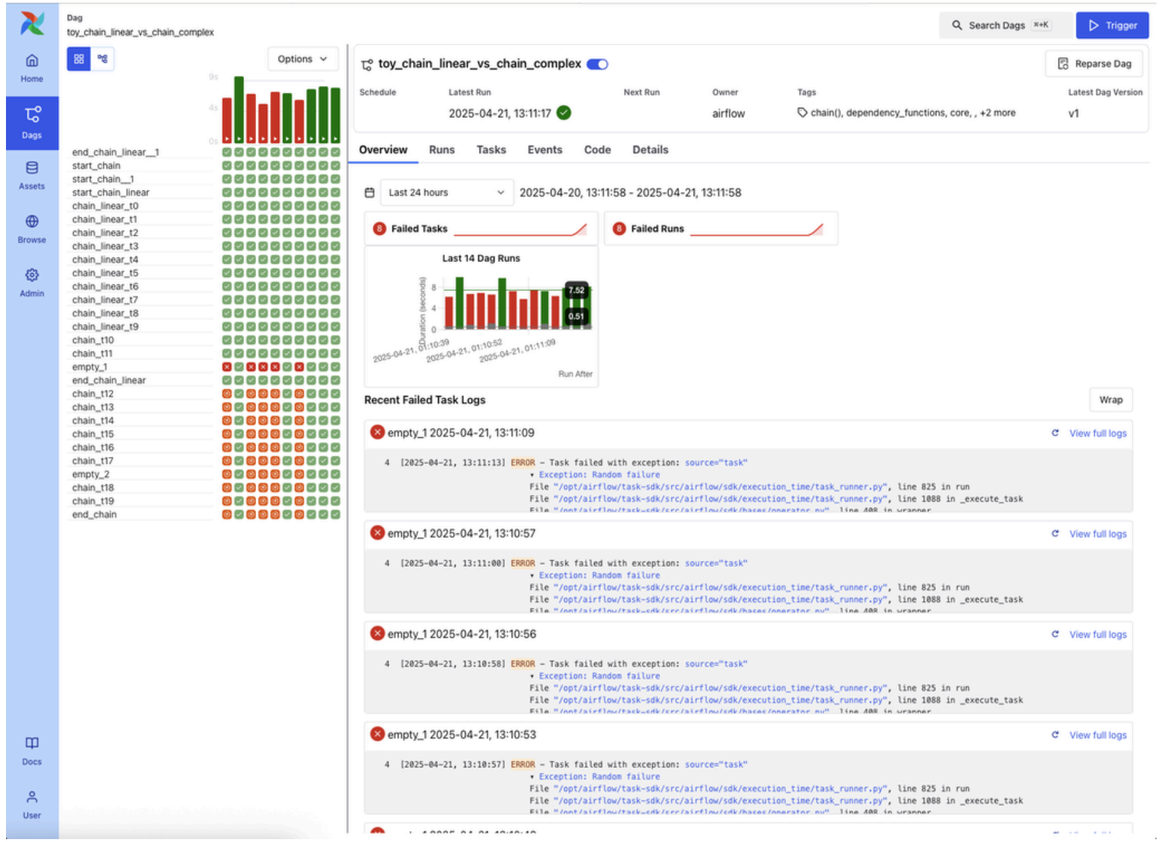
Apache Airflow automates machine learning pipelines.

Why to use ?

Carbon market data updates regularly and models must retrain automatically

How it helps ?

Airflow schedules daily/monthly data loading, retraining, and forecast generation for carbon prices without manual work.



Docker – Environment Management

What is it ?

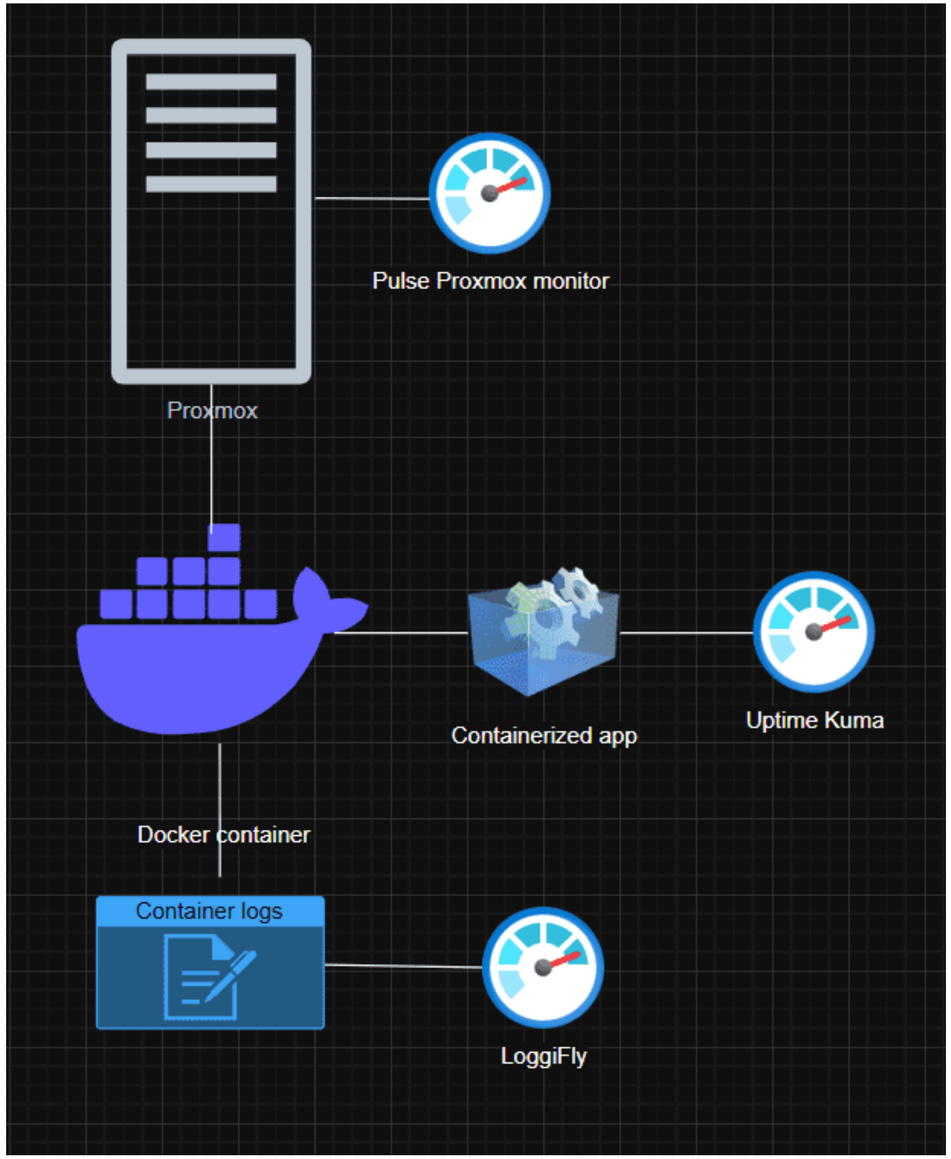
Docker packages code, libraries, and dependencies together.

Why to use ?

To avoid “works on my system” problems.

How it helps ?

The carbon price forecasting system runs the same on laptop, lab PC, and cloud server.



Weights & Biases (W&B) – Training Monitoring

What is it ?

W&B monitors model training and system usage.

Why to use ?

To observe model performance in real time.

How it helps ?

While training the carbon forecasting model, W&B shows accuracy, loss, CPU, and memory usage.

Evidently AI – Data Drift Detection

What is it ?

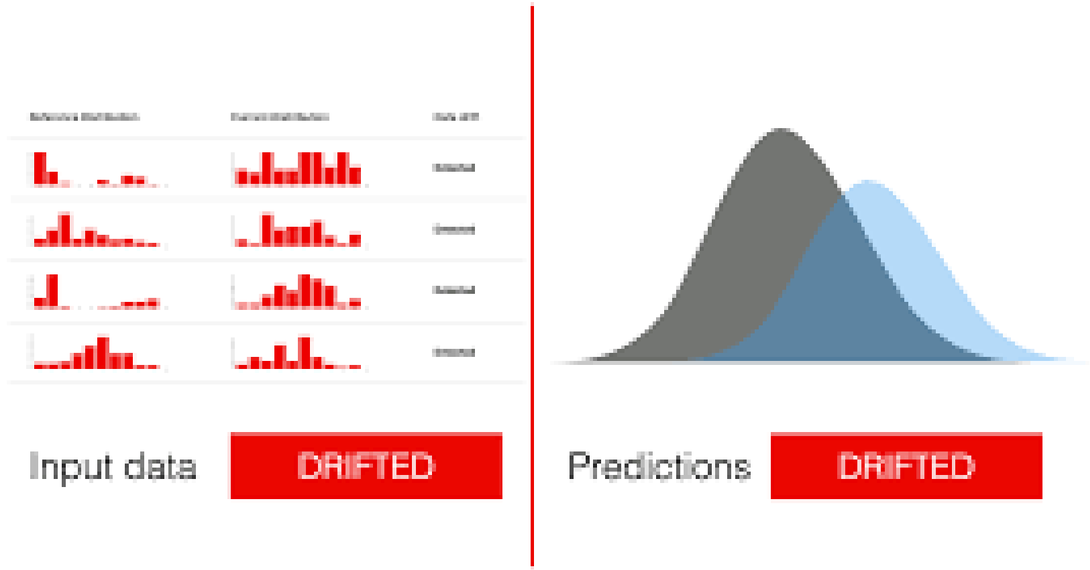
Evidently AI detects changes in data patterns.

Why to use ?

Carbon prices change due to policy and market shifts.

How it helps ?

If new carbon trading data behaves differently than past data, Evidently alerts that the model needs retraining.



GitHub – Code Version Control

What is it ?

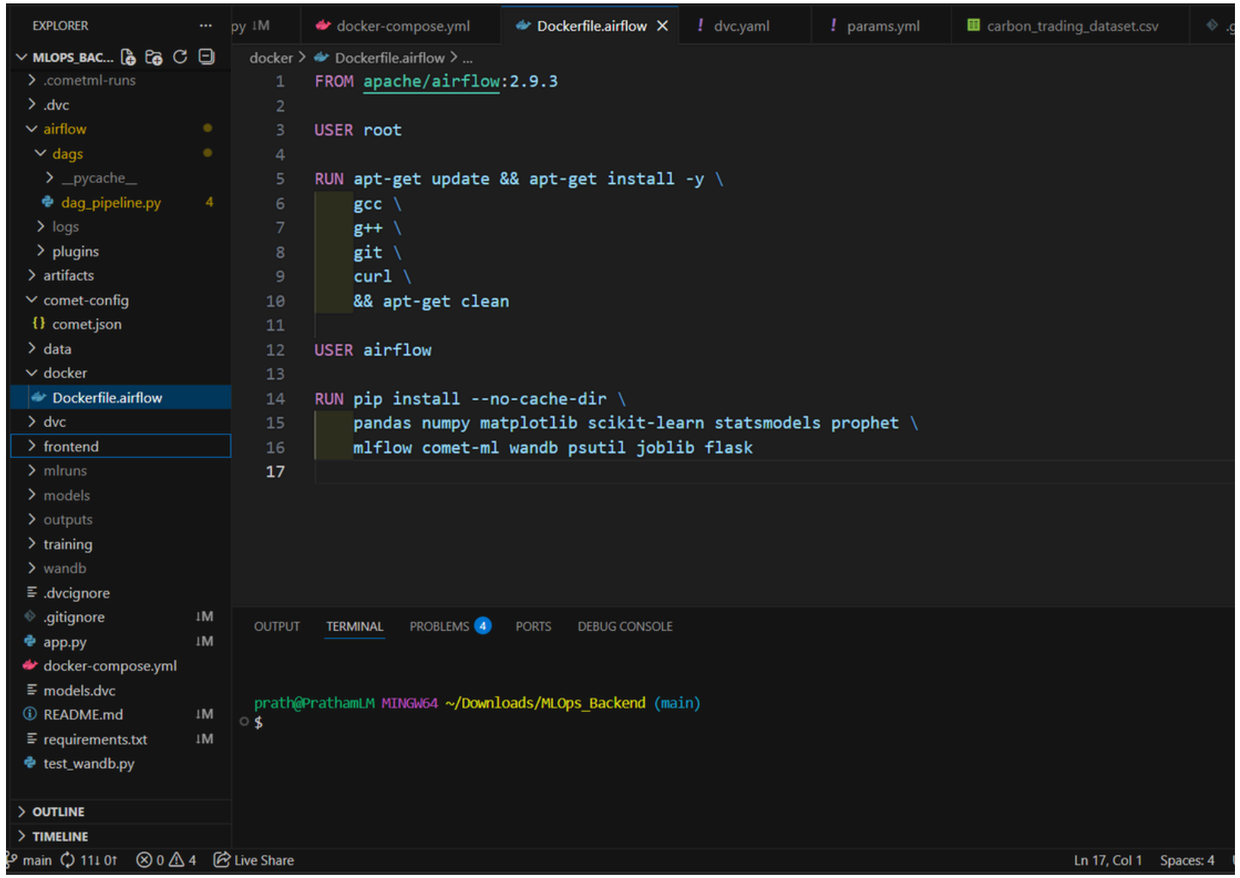
GitHub stores and manages project code versions.

Why to use ?

To track changes and collaborate safely.

How it helps ?

If a code update breaks carbon price prediction, GitHub allows rolling back to a stable version.



DVC – Data & Model Versioning

What is it ?

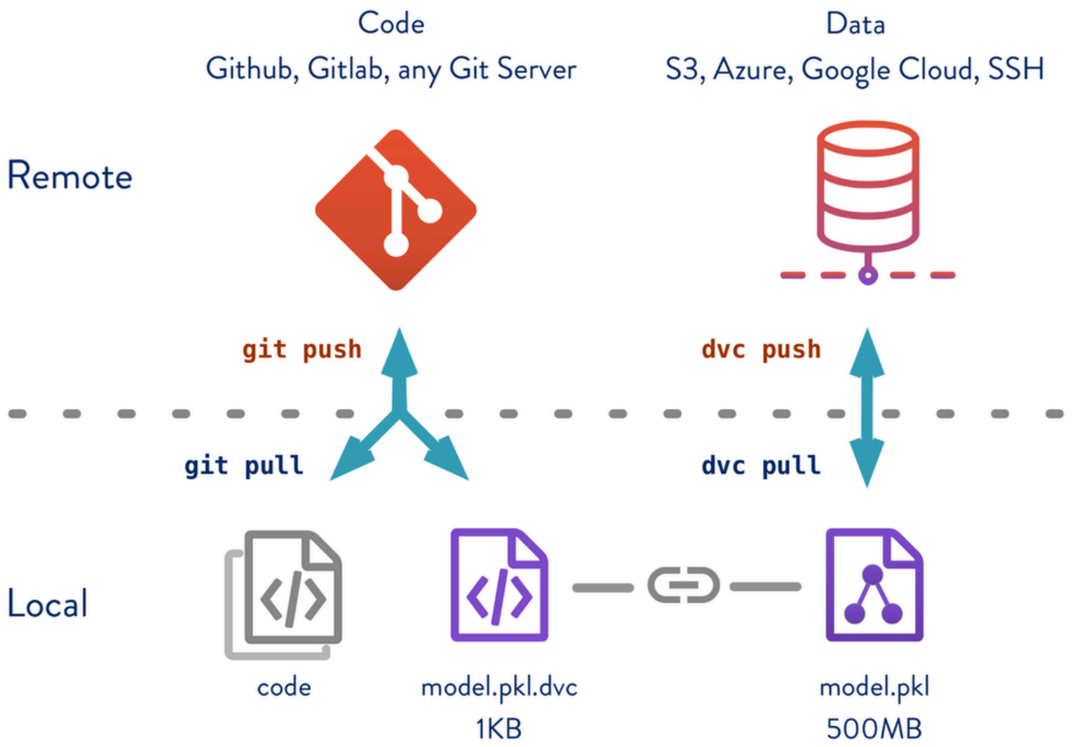
DVC tracks datasets and trained models.

Why to use ?

Machine learning depends on both code and data.

How it helps ?

When carbon trading data is updated, DVC ensures the correct dataset and model versions are linked and reproducible.



Render – Cloud Deployment and Model Hosting

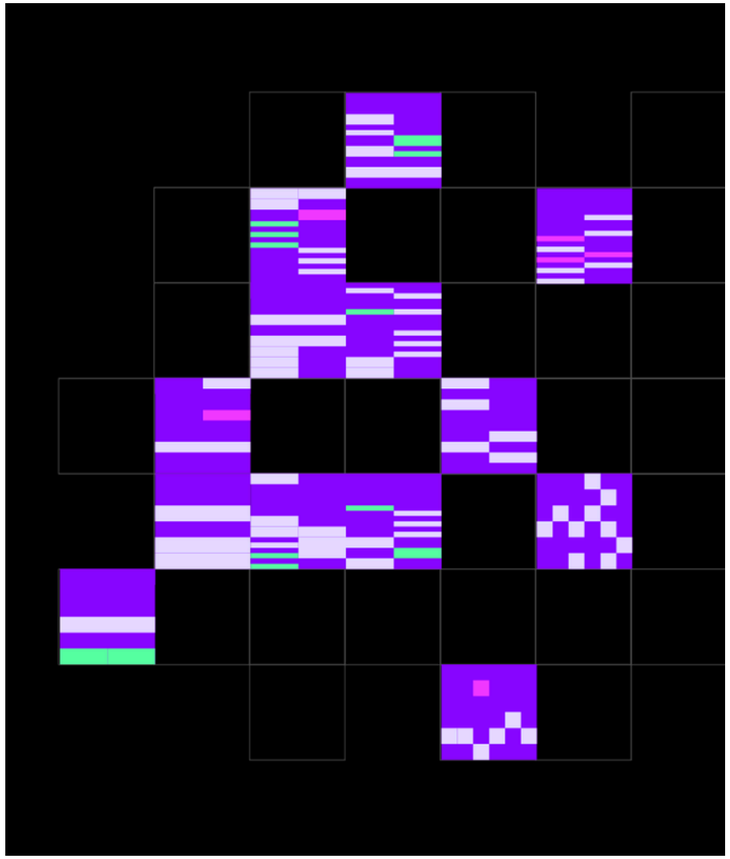
Render DOCS

What is it ?

Render is a cloud platform used to deploy, manage, and scale web services, APIs, and background jobs directly from code repositories.

Why to use ?

Carbon trading price forecasting models must be accessible as a live service for prediction, monitoring, and visualization. Render simplifies deployment without complex cloud configuration.



How it helps ?

Once deployed on Render, stakeholders can access real-time or scheduled carbon price forecasts through an API endpoint, without running the models locally.

	Component	MLOPs Tools
1	Data Management	DVC
2	Experiment Tracking and Version Control	Comet ML, W&B, GitHub
3	Automation and Pipeline Orchestration	Apache Airflow, Docker
4	Model Deployment & Serving	Render, Docker
5	Monitoring & Governance and collaboration	Evidently AI,W&b, GitHub

- **Data Management**
  - Data Processing: We utilized Pandas and NumPy for loading the carbon trading dataset (dataset.csv), handling missing values, and preparing time-series data for modeling.
  - Version Control: DVC (Data Version Control) is integrated to track large dataset files and model artifacts (models.dvc), ensuring that data changes are versioned alongside code in Git.

- **Experiment Tracking and Version Control**

Multi-Platform Tracking: The project implemented a robust tracking system using three major platforms:

  - Comet ML: Used for auto-logging full experiment details, including system metrics and code snapshots. Configured via comet-config/comet.json.
  - MLflow: Sets up a local tracking URI (./mlruns) to log experiments like "Carbon_Forecasting_Inference".
  - Weights & Biases (W&B): Logs inference runs, interactive plots (forecasts, system metrics), and feature importance graphs.
  - Code Versioning: Git manages the source code history, while DVC handles the binary assets.

- **Automation and Pipeline Orchestration**
  - Workflow Orchestration: Apache Airflow (found in airflow/ directory) is used to schedule and automate training and inference pipelines, ensuring reproducible workflows.
  - Container orchestration: Docker Compose (docker-compose.yml) defines the multi-container environment, coordinating the services required for the application and its dependencies.

- **Model Deployment & Serving**
  - Containerization: Docker packages the application, its dependencies (from requirements.txt), and the environment into a portable image. This guarantees that the model runs consistently across development and production environments.
  - Cloud Hosting: Render is utilized as the cloud platform (PaaS) to deploy the Dockerized Flask application. It connects directly to the Git repository/registry, automating builds and deployments.
  - API Framework: The application is built using Flask, exposing endpoints like /predict to serve valid JSON responses for carbon price forecasts.

- **Monitoring & Governance and Collaboration**
  - Statistical Monitoring (Evidently AI): Implements automated drift detection to identify "Model Decay." It compares real-time carbon market distributions against training baselines to detect Data Drift (input changes) and Target Drift (volatility shifts).
  - Visual Monitoring: Live plots of system resources and model forecasts are generated and sent to WandB dashboards for remote monitoring.
  - Governance: The combination of Git (code), DVC (data/models), and Experiment Tracking logs ensures full auditability of who ran what model, when, and with what data.